

Explain *code optimization techniques in detail*. Code optimization is the process of improving the intermediate code or target code to make it more efficient without changing its output. It improves: >Execution speed //>Memory usage >Power efficiency Objectives of Code Optimization: 1. Reduce execution time //2. Minimize memory usage // 3.Eliminate redundant computations 4.Improve overall program performance Types of Code Optimization 1.Machine Independent // 2.Machine Dependent Optimization 1. Machine Independent Optimization These techniques do not depend on machine architecture. Important Techniques: 1. Constant Folding: Compile-time evaluation of constant expressions. // Example: // a = 5 + 3 → a = 8 2. Constant Propagation: Replace variables having constant values. // Example: // a = 10 // b = a + 5 → b = 10 + 5 3. Common Subexpression Elimination (CSE): Avoid recomputing same expression. // Example: x = a + b // y = a + b // → y = x 4. Dead Code Elimination: Removes unused or unreachable code. // Example: // x = 10 // x = 20 (first statement is useless) 5. Loop Optimization: Improves performance of loops. (a) Loop Invariant Code Motion: Moves constant expressions outside loop. // Example: // for(i=0;i<n;i++){ x = a + b // } → x = a + b // for(i=0;i<n;i++){ } (b) Strength Reduction: Replace costly operations with cheaper ones. // Example: // x = y * 2 → x = y + y	Explain *runtime environment of compiler*. >> The runtime environment of a compiler refers to the environment in which a compiled program executes. It includes the organization of memory, storage allocation, and management of variables and functions during program execution. It is responsible for efficient execution of the target program. Functions of Runtime Environment: 1. Allocation of memory 2.Management of function calls 3.Handling of variables (local & global) 4.Parameter passing 5.Supporting recursion 6.Maintaining execution order Components of Runtime Environment 1. Storage Allocation It deals with how memory is assigned to variables and functions. Types of Storage Allocation (a) Static Allocation: Memory allocated at compile time > Fixed size // Example: Global variables// Fast access No dynamic behavior (b) Stack Allocation: Memory allocated at runtime using stack Used for function calls Supports recursion: Efficient // ❌ Limited size (c) Heap Allocation: Memory allocated dynamically at runtime >> Used for dynamic structures Example: malloc(), new // ✓ Flexible // ❌ Slower and complex 2. Activation Record (Stack Frame) An activation record is a block of memory used to store information about a function call.	Explain *lexical analysis in detail* with token recognition. Lexical Analysis is the first phase of a compiler. It reads the source program as a sequence of characters and converts it into a sequence of tokens. // It is also called a scanner. Functions of Lexical Analyzer: 1. Converts input program into tokens 2.Removes whitespace and comments 3. Identifies lexical errors 4. Creates entries in symbol table 5. Passes tokens to the parser Basic Terminology// Token: A token is a valid unit of a program. Example: // int a = 10; // → Tokens: int, a, =, 10 Lexeme: Actual sequence of characters forming a token. Example: // Token: identifier // Lexeme: a Pattern: Rule used to define a token. Example:// Identifier → [a-zA-Z][a-zA-Z0-9]* Process of Lexical Analysis Source Code → Lexical Analyzer → Tokens → Syntax Analyzer Token Recognition: Token recognition is the process of identifying tokens from input using patterns and finite automata. Techniques Used: 1. Regular Expressions: Used to define patterns for tokens //Examples: // Identifier → letter (letter digit)* Number → digit+ 2. Finite Automata (DFA): Used to recognize tokens efficiently // Converts regular expressions into DFA Example for identifier: // Start → letter → (letter/digit loop) Example of Token Recognition Input: // a = b + 10																								
Consider the expression: (c(a + a) + (a + a)) + ((a + a) + (a + a)) Construct the Directed Acyclic Graph (DAG) for it. Step 1: Identify Common Subexpressions The expression (a + a) appears multiple times So we create only ONE node for: // (a + a) Let: // t1 = a + a Step 2: Rewrite Expression: // (c * t1 + t1) + (t1 + t1) Step 3: Create DAG Nodes: Leaf Nodes: // a // c Internal Nodes: 1. t1 = a + a // 2. t2 = c * t1 // 3. t3 = t2 + t1 // 4. t4 = t1 + t1 // 5. Final = t3 + t4 DAG Representation <pre> (+) / \ (+) (+) / \ / \ (*) t1 t1 t1 / \ c t1 (+) / \ a a </pre>	Construct *SLR parsing table* for given grammar. SLR (Simple LR) parser is a type of bottom-up parser that uses: LR(0) items // FOLLOW sets It is simpler than LALR and CLR. Steps to Construct SLR Parsing Table Step 1: Augment the Grammar Add new start symbol: // S' → S Step 2: Construct LR(0) Items: Find closure() // Find goto() // Build canonical collection of LR(0) items Step 3: Construct DFA (State Diagram): Each state = set of LR(0) items // Transitions based on grammar symbols Step 4: Compute FOLLOW Sets Used for reduce actions Step 5: Construct Parsing Table ACTION Table: Shift (S) //Reduce (R) // Accept (acc) GOTO Table: For non-terminals Example: Given Grammar: S → CC // C → cC // C → d Step 1: Augmented Grammar S' → S // S → CC // C → cC // C → d Step 2: LR(0) Items I₀: // S' → •S // S → •CC // C → •cC // C → •d I₁ (goto(I₀, S)): S' → S• I₂ (goto(I₀, C)): // S → C•C // C → •cC // C → •d I₃ (goto(I₀, c)): C → c•C // C → •cC // C → •d I₄ (goto(I₀, d)): // C → d• I₅ (goto(I₁, C)): //S → CC• //I₆ (goto(I₁, C)): // C → cC•	Construct *LALR parsing table*. LALR (Look-Ahead LR) parser is a type of bottom-up parser obtained by merging LR(1) states having same LR(0) items. It is: // More powerful than SLR // More compact than CLR Idea of LALR Parsing Start with LR(1) items // Merge states with same core (LR(0) items) // Combine their lookahead symbols Then construct ACTION and GOTO table Steps to Construct LALR Parsing Table Step 1: Augment the Grammar Add a new start symbol: // S' → S Step 2: Construct LR(1) Items : Compute closure and goto // Build canonical LR(1) collection Step 3: Merge States: Find states with same LR(0) core // Merge them // Combine their lookahead sets This reduces number of states Step 4: Construct Parsing Table ACTION Table: Shift // Reduce // Accept GOTO Table: For non-terminals Example // Given Grammar: S → L = R // S → R // L → * R // L → id // R → L Step 1: Augmented Grammar: S' → S Step 2: LR(1) Items : (Construct full LR(1) states – usually shown in exam briefly) // Step 3: Merge States 👉 Example: // If two states have: L → id • , a // L → id • , b // Merge → // L → id • , {a, b}																								
Generate *Three Address Code for control structures*. Definition of Three Address Code (TAC) Three Address Code is an intermediate code representation in which each instruction contains at most three addresses. General form: // x = y op z Used in compiler for: Easy optimization // Simple code generation Control Structures in TAC: If statement // IF-Else statement // While loop // Do-While loop 1. TAC for IF Statement: Syntax://if (condition)// S1; TAC Form: if condition goto L1 // goto L2 // L1: S1 // L2: Example: if (a < b) // x = y + z; TAC: if a < b goto L1 // goto L2 // L1: x = y + z // L2: 2. TAC for IF-ELSE Statement Syntax: if (condition) // S1 // else // S2; TAC Form: if condition goto L1 goto L2 // L1: S1 // goto L3 // L2: S2 // L3: Example : if (a > b) // x = a; // else // x = b; TAC: if a > b goto L1 goto L2 // L1: x = a // goto L3 // L2: x = b // L3: 3. TAC for WHILE Loop: Syntax: while (condition) // S1; TAC Form: L1: if condition goto L2 goto L3 // L2: S1 // goto L1 // L3: Example : while (i < 10) // i = i + 1; TAC: L1: if i < 10 goto L2 // goto L3 L2: i = i + 1 // goto L1 // L3: 4. TAC for DO-WHILE Loop Syntax: // do { // S1; // } while (condition); TAC Form: L1: S1 // if condition goto L1 Example: do { // x = x + 1; // } while (x < 5); TAC: L1: x = x + 1 // if x < 5 goto L1 5. TAC for Nested IF (Important) Example: if (a > b) // if (c > d) // x = 1; TAC: if a > b goto L1 // goto L3 // L1: if c > d goto L2 // goto L3 // L2: x = 1 // L3: // Advantages of TAC: Simple and easy to understand // Helps in optimization // Machine-independent representation // Disadvantages: Large number of instructions Uses many temporary variables	What is the role of parsing? Explain different types of parsing in compiler design. 👍 Role of Parsing (Syntax Analysis) Parsing is the second phase of a compiler. It is also called Syntax Analysis. 🏆 Definition: Parsing is the process of analyzing a sequence of tokens to determine whether it follows the grammar rules (CFG) of the programming language. 🎯 Main Roles of Parsing 1.Checks Syntax Correctness: Ensures program follows language rules. 2.Buils Parse Tree / Syntax Tree: Shows hierarchical structure of program. 3.Detects Syntax Errors: Missing semicolon, brackets, etc. 4.Provides Structure for Next Phases: Helps semantic analysis and code generation. Types of Parsing: Parsing is mainly divided into two types: 🔹 1. Top-Down Parsing 🏆 Definition: Top-down parsing starts from the start symbol and tries to generate the input string. Direction: Root → Leaves // Types of Top-Down Parsing (a) Recursive Descent Parsing: Uses recursive functions. >May require backtracking. // Simple but less efficient. (b) Predictive Parsing (LL(1)): >Uses no backtracking. >Uses FIRST and FOLLOW sets. >Uses parsing table. Most efficient top-down method. Advantages: Easy to understand and implement. Disadvantages: Cannot handle left recursion. Limited grammar support. 🔹 2. Bottom-Up Parsing 🏆 Definition: Bottom-up parsing starts from the input string and reduces it to the start symbol. Direction: Leaves → Root // Types of Bottom-Up Parsing (a) Shift-Reduce Parsing: >Uses shift and reduce operations. Builds parse tree in reverse. (b) LR Parsing: Most powerful parsing method. Types: SLR (Simple) // CLR (Canonical) // LALR (Look Ahead) Advantages: Handles more complex grammars. No backtracking required. Disadvantages: Difficult to implement. Parsing tables are large.	Explain *storage allocation strategies* in detail. Storage allocation strategies refer to the methods used by a compiler to allocate memory to variables and program data during execution. It is a part of the runtime environment. Objectives: >Efficient use of memory // >Fast access to variables >Support for recursion and dynamic data >Proper management of program execution Types of Storage Allocation Strategies: There are three main types: Static Allocation // Stack Allocation // Heap Allocation 1. Static Storage Allocation: Memory is allocated at compile time and remains fixed throughout program execution. Features: Fixed memory size // Allocated before execution Address of variables does not change // Example: int x = 10; Advantages: ✓ Simple and fast // ✓ No runtime overhead Disadvantages: No support for recursion // No dynamic memory allocation // Wastage of memory 2. Stack Storage Allocation: Memory is allocated and deallocated at runtime using a stack (LIFO). // Features: Used for function calls // Each function gets an activation record // Supports recursion // Stack Representation Top → Function C // Function B // Function A // Bottom // Example: void fun() { // int a; // } Advantages : Efficient memory usage // Supports recursion Automatic allocation and deallocation // Disadvantages: Limited memory size // Cannot allocate complex dynamic structures 3. Heap Storage Allocation: Memory is allocated dynamically at runtime from the heap. Features : Memory allocated as needed (// Used for dynamic data structures / Example : int *p = (int*)malloc(sizeof(int)); Advantages: Flexible memory usage // Supports dynamic data structures (linked list, tree) // Disadvantages: Slower than stack Complex management // Memory leaks possible Comparison of Storage Allocation Strategies <table><tr><th>Feature</th><th>Static</th><th>Stack</th><th>Heap</th></tr><tr><td>Allocation Time</td><td>Compile time</td><td>Runtime</td><td>Runtime</td></tr><tr><td>Memory Size</td><td>Fixed</td><td>Dynamic (limited)</td><td>Dynamic</td></tr><tr><td>Recursion</td><td>Not supported</td><td>Supported</td><td>Supported</td></tr><tr><td>Speed</td><td>Fast</td><td>Fast</td><td>Slow</td></tr><tr><td>Complexity</td><td>Simple</td><td>Medium</td><td>Complex</td></tr></table>	Feature	Static	Stack	Heap	Allocation Time	Compile time	Runtime	Runtime	Memory Size	Fixed	Dynamic (limited)	Dynamic	Recursion	Not supported	Supported	Supported	Speed	Fast	Fast	Slow	Complexity	Simple	Medium	Complex
Feature	Static	Stack	Heap																							
Allocation Time	Compile time	Runtime	Runtime																							
Memory Size	Fixed	Dynamic (limited)	Dynamic																							
Recursion	Not supported	Supported	Supported																							
Speed	Fast	Fast	Slow																							
Complexity	Simple	Medium	Complex																							

Output Tokens: <table><thead><tr><th>Lexeme</th><th>Token Type</th></tr></thead><tbody><tr><td>a</td><td>Identifier</td></tr><tr><td>=</td><td>Operator</td></tr><tr><td>b</td><td>Identifier</td></tr><tr><td>+</td><td>Operator</td></tr><tr><td>10</td><td>Number</td></tr></tbody></table> Role of Symbol Table Stores information about identifiers: Name // Type // Scope Lexical Errors Examples: Invalid symbols (@, # etc.) // Wrong identifiers (1abc) Handled by: Skipping characters // Error messages Input Buffering: Used for efficient reading of input // Uses two-buffer scheme LEX Tool: Automatic tool to generate lexical analyzer // Uses regular expressions Advantages: Simplifies syntax analysis // Improves efficiency // Removes unnecessary data Disadvantages: Limited error handling // Cannot detect syntax errors	Lexeme	Token Type	a	Identifier	=	Operator	b	Identifier	+	Operator	10	Number	Structure of Activation Record Return Address → Where control returns after function Parameters → Input values // Local Variables → Variables inside function // Temporary variables Control Link → Points to previous function Access Link → Used for non-local variables 3. Parameter Passing Methods Call by Value: Copy of value is passed ✓ Safe // ✗ No modification of original Call by Reference: Address of variable is passed ✓ Changes reflect // ✗ Risky 4. Access to Variables Local Variables: Stored in activation record // Accessed directly Non-local Variables: Accessed using access link or static link 5. Symbol Table: A data structure used by compiler to store information about variables. Stores: >Variable name > Type > Scope > Memory location Data Structures Used: Hash table > Linked list > Tree 6. Heap Management: Handles dynamic memory allocation Allocates and deallocates memory during runtime Advantages of Runtime Environment ✓ Efficient memory management // ✓ Supports recursion // ✓ Enables dynamic allocation // ✓ Improves execution control Disadvantages // Complex implementation Memory overhead // Risk of memory leaks (heap)	(c) Loop Unrolling: Reduce loop overhead by repeating statements. 6. Copy Propagation: Replace variables with assigned values. Example: // a = b // c = a → c = b 7. Algebraic Simplification: Simplify expressions using algebra rules. // Example: // x = x + 0 → x = x // x = x * 1 → x = x 8. Peephole Optimization: Looks at small sets of instructions and replaces inefficient ones. Example: // MOV R1, 0 // ADD R1, X → MOV R1, X 2. Machine Dependent Optimization: These depend on hardware architecture. Techniques 1. Register Allocation: Efficient use of CPU registers. 2. Instruction Scheduling: Rearranges instructions to improve speed. 3. Use of Special Instructions: Uses machine-specific instructions for efficiency. Advantages of Code Optimization : Faster execution Reduced memory usage // Better CPU utilization Disadvantages: Increases compilation time Complex implementation // May increase code size (in some cases)																																																																									
Lexeme	Token Type																																																																																						
a	Identifier																																																																																						
=	Operator																																																																																						
b	Identifier																																																																																						
+	Operator																																																																																						
10	Number																																																																																						
Step 4: LALR Parsing Table Final Table Format <table><thead><tr><th>State</th><th>Id</th><th>*</th><th>\$</th><th>S</th><th>L</th><th>R</th></tr></thead><tbody><tr><td>0</td><td>S5</td><td>S4</td><td></td><td>1</td><td>2</td><td>3</td></tr><tr><td>1</td><td></td><td></td><td>acc</td><td></td><td></td><td></td></tr></tbody></table> Important Points: LALR merges states → fewer states than LR(1) // Same power as LR(1) in most cases Used in tools like YACC Advantages: Smaller parsing table // Efficient memory usage // Widely used in practice // Disadvantages: Complex construction // May introduce conflicts in some cases Difference: SLR vs LALR vs CLR <table><thead><tr><th>Feature</th><th>SLR</th><th>LALR</th><th>CLR</th></tr></thead><tbody><tr><td>Power</td><td>Low</td><td>Medium</td><td>High</td></tr><tr><td>States</td><td>Few</td><td>Moderate</td><td>Large</td></tr><tr><td>Accuracy</td><td>Less</td><td>More</td><td>Highest</td></tr></tbody></table>	State	Id	*	\$	S	L	R	0	S5	S4		1	2	3	1			acc				Feature	SLR	LALR	CLR	Power	Low	Medium	High	States	Few	Moderate	Large	Accuracy	Less	More	Highest	Step 3: FOLLOW Sets FOLLOW(S) = { \$ } // FOLLOW(C) = { c, d, \$ } Step 4: SLR Parsing Table <table><thead><tr><th>State</th><th>c</th><th>d</th><th>\$</th><th>S</th><th>C</th></tr></thead><tbody><tr><td>0</td><td>S3</td><td>S4</td><td></td><td>1</td><td>2</td></tr><tr><td>1</td><td></td><td></td><td>acc</td><td></td><td></td></tr><tr><td>2</td><td>S3</td><td>S4</td><td></td><td></td><td>5</td></tr><tr><td>3</td><td>S3</td><td>S4</td><td></td><td></td><td>6</td></tr><tr><td>4</td><td>R3</td><td>R3</td><td>R3</td><td></td><td></td></tr><tr><td>5</td><td></td><td></td><td>R1</td><td></td><td></td></tr><tr><td>6</td><td>R2</td><td>R2</td><td>R2</td><td></td><td></td></tr></tbody></table> Productions Numbering 1. S → CC // 2. C → cC // 3. C → d Explanation: >S3, S4 → Shift R1, R2, R3 → Reduce // >acc → Accept Advantages Simple to construct // Requires less memory Disadvantages: Less powerful than LR(1) // May produce conflicts	State	c	d	\$	S	C	0	S3	S4		1	2	1			acc			2	S3	S4			5	3	S3	S4			6	4	R3	R3	R3			5			R1			6	R2	R2	R2			Explanation Only one node is created for (a + a) → reused everywhere This eliminates redundant computation DAG shares common subexpressions Optimized Computation: // t1 = a + a // t2 = c * t1 t3 = t2 + t1 // t4 = t1 + t1 // result = t3 + t4 ✓ Advantages of DAG in this Example ✓ Avoids recomputing (a + a) multiple times ✓ Reduces number of operations ✓ Improves efficiency Consider the following basic block and construct the DAG for it: <pre>t1 = a + b // t2 = c + d // t3 = t2 * c // t4 = t1 - t3</pre>
State	Id	*	\$	S	L	R																																																																																	
0	S5	S4		1	2	3																																																																																	
1			acc																																																																																				
Feature	SLR	LALR	CLR																																																																																				
Power	Low	Medium	High																																																																																				
States	Few	Moderate	Large																																																																																				
Accuracy	Less	More	Highest																																																																																				
State	c	d	\$	S	C																																																																																		
0	S3	S4		1	2																																																																																		
1			acc																																																																																				
2	S3	S4			5																																																																																		
3	S3	S4			6																																																																																		
4	R3	R3	R3																																																																																				
5			R1																																																																																				
6	R2	R2	R2																																																																																				
Explain *syntax-directed translation in detail*. Syntax-Directed Translation (SDT) is a method in compiler design where semantic rules (actions) are attached to the grammar productions to perform translation. It is used to: Generate intermediate code // Build syntax trees // Perform type checking Basic Idea: >Each grammar rule is associated with semantic actions > These actions are executed during parsing > Translation is done based on syntax structure Example: Grammar: // E → E1 + T // E → T With semantic rule: // E.val = E1.val + T.val Here, .val is an attribute Components of SDT: 1. Attributes: Attributes store information about grammar symbols. Types of Attributes: (a) Synthesized Attributes: Computed from child nodes // Flow: bottom-up Example: // E.val = E1.val + T.val (b) Inherited Attributes: Passed from parent or sibling nodes Flow: top-down // Example: // T.type = E.type 2. Syntax-Directed Definition (SDD): An SDD is a context-free grammar with attributes and semantic rules. Types of SDD: 1. S-Attributed Definition: Uses only synthesized attributes // Evaluated in bottom-up parsing ✓ Easy to implement 2. L-Attributed Definition: Uses both inherited and synthesized attributes // Evaluated in top-down parsing // ✓ More powerful 3. Syntax Tree Construction: SDT helps in building: >> Parse Tree >> Abstract Syntax Tree (AST) Example: // E → E + T // Tree: E \ / E T 4. Intermediate Code Generation: SDT is used to generate Three Address Code (TAC). Example: // E → E1 + T E.addr = newtemp() // E.code = E1.code T.code E.addr = E1.addr + T.addr Applications of SDT: Expression evaluation // Type checking Intermediate code generation // Syntax tree construction Advantages: Systematic translation// Easy to implement with parser// Supports automatic code generation Disadvantages: Complex for large grammars // Difficult to debug	Q. What is a Compiler? Explain the phases of a compiler with a neat block diagram. A compiler is a system software that translates a program written in a high-level language (like C, Java) into machine-level language (binary code) so that it can be executed by the computer. It checks for errors and converts the entire program at once. ✓ Phases of a Compiler: A compiler works in different steps called phases. Each phase performs a specific task in the translation process. Source Program → Lexical Analysis → Syntax Analysis → Semantic Analysis → Intermediate Code Generation → Code Optimization → Target Code Generation → Target Program Symbol Table (used to store variable info) // Error Handler (handles errors in each phase) 1. Lexical Analysis (Scanner): >First phase of compiler. Converts source code into tokens. // Removes spaces, comments. Example: int a = 10; // → int, a, =, 10 2. Syntax Analysis (Parser) >Checks whether the program follows grammar rules. >Generates parse tree. Example: // Detects missing semicolon error. 3. Semantic Analysis >Checks meaning of program. >Type checking (int, float, etc.) Example: // int a; // a = "hello"; (type error) 4. Intermediate Code Generation Converts program into intermediate code (IC). // >Machine independent. Example: a = b + c // → t1 = b + c // → a = t1 5. Code Optimization: Improves code efficiency. Removes unnecessary instructions. Example: // a = b + 0 → a = b 6. Target Code Generation Converts optimized code into machine code. Final output is executable program. Symbol Table: Stores information about variables: Name // Type // Scope Error Handling: Detects and reports errors in each phase: Lexical // Syntax // Semantic errors	Explain *DAG representation of basic block*. Definition of Basic Block: A basic block is a sequence of consecutive statements in a program with: // Single entry point // Single exit point // No branching in between DAG (Directed Acyclic Graph) is a graphical representation used to represent expressions in a basic block. It helps in code optimization by identifying: Common subexpressions // >> Redundant computations Purpose of DAG: > Eliminate repeated calculations Optimize expressions // > Improve code efficiency Structure of DAG: Leaves (Nodes) → Represent variables or constants // > Interior Nodes → Represent operators (+, -, *, /) > Edges → Show dependency between operands Example: Given Basic Block: // a = b + c // d = b + c // e = a + d Step-by-Step DAG Construction: > Create nodes for b and c > Create node for b + c // > Assign it to a and d (same node reused) Create node for a + d DAG Representation (Explanation): > One node represents b + c & e It is shared by both a and d // > This avoids recomputation Final optimized idea: // t1 = b + c // a = t1 // d = t1 // e = t1 + t1 Advantages of DAG: > Eliminates common subexpressions > Reduces redundant calculations > Improves execution speed Helps in further optimization Disadvantages: Complex to construct // > Works only within a basic block // > Not suitable for global optimization Applications of DAG: Code optimization Compiler design // Expression simplification Write a lex program to count the number of words, lines, and characters in an input string <pre>%{ // #include <stdio.h> // int words = 0; int lines = 0; // int chars = 0; // } // %} // %} \n { lines++; chars++; } // [\t]+ { chars += yyleng; } [^\n]+ { words++; chars += yyleng; } // %} int main() // { // yylex(); printf("Number of lines = %d\n", lines); printf("Number of words = %d\n", words); printf("Number of characters = %d\n", chars); return 0; } // int yywrap() // { // return 1; // }</pre>																																																																																					

Explain top-down and bottom-up parsing techniques in detail. Parsing is the process of analysing a string of tokens to check whether it follows the grammar of a language. It builds a parse tree for the given input. There are two main parsing techniques: 1.Top-Down Parsing // 2.Bottom-Up Parsing 1. Top-Down Parsing: Top-down parsing starts from the start symbol (root) and tries to derive the input string step by step. It constructs the parse tree from root to leaves. Working: >Begins with the start symbol (S) >Applies production rules >Expands non-terminals until the input string is obtained Example: Grammar: $S \rightarrow aA$ // $A \rightarrow b$ // Input: ab Derivation: $S \rightarrow aA \rightarrow ab$ Types of Top-Down Parsing (a) Recursive Descent Parsing: Uses recursive procedures for each non-terminal. > May involve backtracking. Disadvantage: Time-consuming due to backtracking. (b) Predictive Parsing (LL(1)): Uses no backtracking. Uses FIRST and FOLLOW sets. > Uses parsing table. 👉 Advantage: Fast and efficient. Advantages: Easy to understand and implement Suitable for simple grammars Disadvantages: Cannot handle left recursion > Limited grammar support // >Backtracking (in some cases) 2. Bottom-Up Parsing: Bottom-up parsing starts from the input string and reduces it to the start symbol. It constructs the parse tree from leaves to root. Working:>Input string is scanned // > Substrings are reduced to non-terminals //> Continue until only start symbol remains Example // Grammar: $S \rightarrow aA$ // $A \rightarrow b$ Input: ab // Steps: $ab \rightarrow aA \rightarrow S$ Types of Bottom-Up Parsing ♦ (a) Shift-Reduce Parsing: Uses two operations: Shift → Move input to stack Reduce → Replace substring using production ♦ (b) LR Parsing**: Most powerful technique. Types: SLR (Simple LR) // CLR (Canonical LR) // LALR (Look-Ahead LR) Advantages: Handles complex grammars No backtracking required // >More powerful than top-down ❌ Disadvantages: Difficult to understand Complex implementation // >Large parsing tables 📦 Difference Between Top-Down and Bottom-Up Parsing <table> <tr> <th>Feature</th><th>Top-Down Parsing</th><th>Bottom-Up Parsing</th></tr> <tr> <td>Direction</td><td>Root → Leaves</td><td>Leaves → Root</td></tr> <tr> <td>Approach</td><td>Expansion</td><td>Reduction</td></tr> <tr> <td>Start Point</td><td>Start symbol</td><td>Input string</td></tr> <tr> <td>Backtracking</td><td>May be required</td><td>Not required</td></tr> <tr> <td>Complexity</td><td>Simple</td><td>Complex</td></tr> <tr> <td>Example</td><td>LL(1)</td><td>LR, SLR</td></tr> </table>	Feature	Top-Down Parsing	Bottom-Up Parsing	Direction	Root → Leaves	Leaves → Root	Approach	Expansion	Reduction	Start Point	Start symbol	Input string	Backtracking	May be required	Not required	Complexity	Simple	Complex	Example	LL(1)	LR, SLR		
Feature	Top-Down Parsing	Bottom-Up Parsing																					
Direction	Root → Leaves	Leaves → Root																					
Approach	Expansion	Reduction																					
Start Point	Start symbol	Input string																					
Backtracking	May be required	Not required																					
Complexity	Simple	Complex																					
Example	LL(1)	LR, SLR																					